

LAB # 06 (B)

Exercise: Select your previous project and Design Test cases for at least 5 features using any of black box strategy [you can choose different techniques to test different features] Attached all printouts and screenshots of application [UI+ application o/p on test data] here. Also attached summary report.

Black Box Testing

Functionality: Registration

Code

```
def register():
    username = reg_username_entry.get().strip()
    password = reg_password_entry.get().strip()
    account_number = reg_account_number_entry.get().strip()
    pin = reg_pin_entry.get().strip()

    if not username or not password or not account_number or not pin:
        messagebox.showerror("Error", "All fields are required!")
        return
    if len(password) < 8:
        messagebox.showerror("Error", "Password must be at least 8 characters long!")
        return
    if not username.isalnum():
        messagebox.showerror("Error", "Username must contain only letters and numbers!")
        return
    if username in users:
        messagebox.showerror("Error", "User already exists!")
        return
    if not account_number.isdigit() or len(account_number) != 10:
        messagebox.showerror("Error", "Account number must be a 10-digit number!")
        return
    if len(pin) != 4 or not pin.isdigit():
        messagebox.showerror("Error", "PIN must be a 4-digit number!")
        return

    users[username] = {"password": password, "account_number": account_number, "pin": pin, "balance": 0}
    messagebox.showinfo("Success", "Registration Successful!")
    reg_username_entry.delete(0, tk.END)
    reg_password_entry.delete(0, tk.END)
```

```
reg_account_number_entry.delete(0, tk.END)
reg_pin_entry.delete(0, tk.END)
```

Summary Report

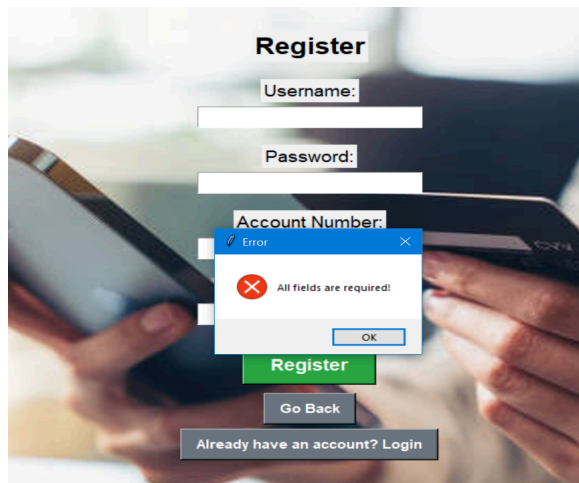
Test Case ID	Test Case Description	Test Steps	Test Data	Expected Result	Actual Result	Status
TC_R01	Register with empty fields	1. Leave all fields empty	Username: ""	Error: All fields are required!	Error: All fields are required!	Pass
		2. Click Register	Password: ""			
			Account: ""			
			PIN: ""			
TC_R02	Password less than 8 characters	1. Enter valid username	Username: test1	Error: Password must be at least 8 characters long!	Error: Password must be at least 8 characters long!	Pass
		2. Enter short password	Password: 123			
		3. Valid account & pin	Account: 1234567890			
		4. Click Register	PIN: 1234			
TC_R03	Username with special characters	1. Enter username with special chars	Username: test!@	Error: Username must contain only letters and numbers!	Error: Username must contain only letters and numbers!	Pass
		2. Valid password, account, pin	Password: password123			
		3. Click Register	Account: 1234567890			
			PIN: 1234			
TC_R04	Account number not 10 digits	1. Enter valid username	Username: test2	Error: Account number must be a 10-digit number!	Error: Account number must be a 10-digit number!	Pass
		2. Valid password	Password: pass12345			
		3. Short account number	Account: 12345			
		4. Click Register	PIN: 1234			
TC_R05	PIN not 4 digits	1. Enter valid username/password/account	Username: test3	Error: PIN must be a 4-digit number!	Error: PIN must be a 4-digit number!	Pass
		2. Invalid PIN	Password: password			
		3. Click Register	Account: 1234567890			
			PIN: 123			
TC_R06	User already exists	1. Register a valid user	Username: javeria	Error: User already exists!	Error: User already exists!	Pass
		2. Try to register again with same username	Password: password123			
			Account: 1234567890			
			PIN: 1234			
TC_R07	Valid Registration	1. Fill all fields correctly	Username: sana	Success: Registration Successful!	Success: Registration Successful!	Pass
		2. Click Register	Password: pass12345			
			Account: 0987654321			
			PIN: 5678			

Class Partitioning

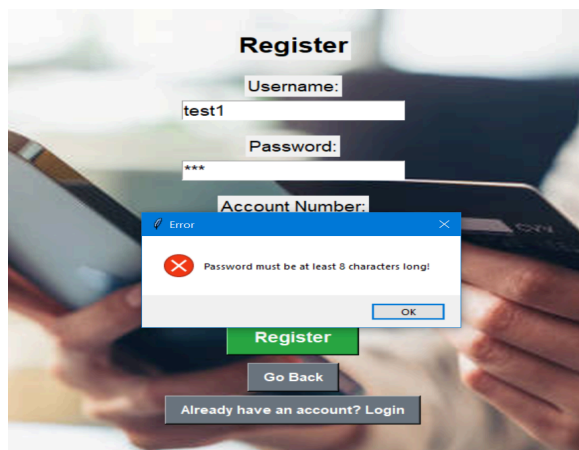
Field	Valid Equivalence Class	Invalid Equivalence Class
Username	Only letters and numbers (e.g., User123)	Empty input, Special characters (e.g., User@123)
Password	8 or more characters (e.g., password123)	Less than 8 characters (e.g., pass12)
Account Number	Exactly 10 digits (e.g., 1234567890)	Not 10 digits (e.g., 12345), Non-numeric characters (e.g., 1234abcd56)
PIN	Exactly 4 digits (e.g., 1234)	Not 4 digits (e.g., 12), Non-numeric (e.g., 12ab)
Duplicate Username	Username not already in system	Username already exists

Test Case

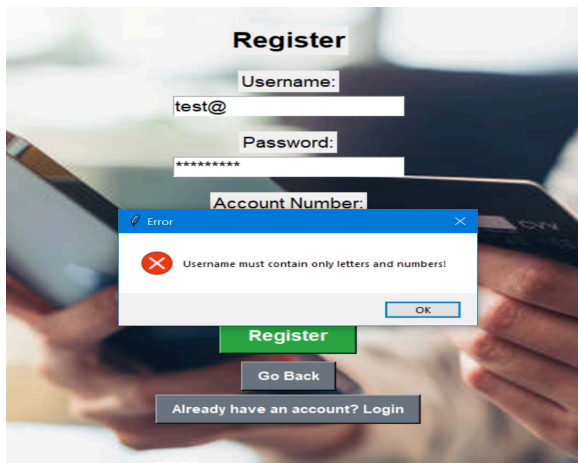
TC-R01



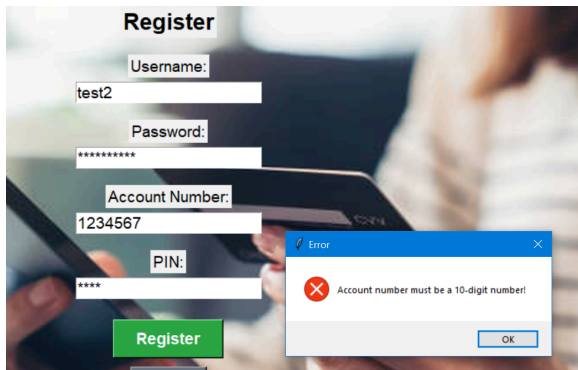
TC-R02



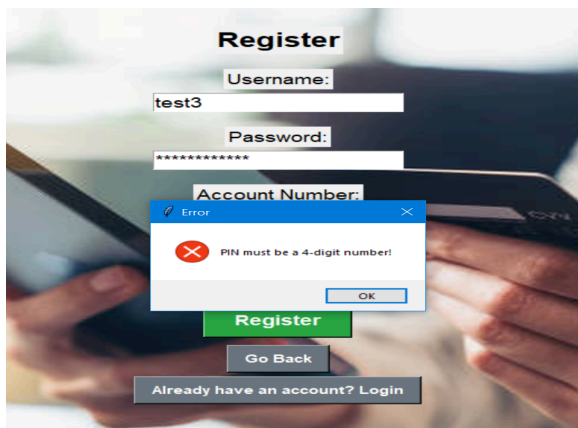
TC-R03



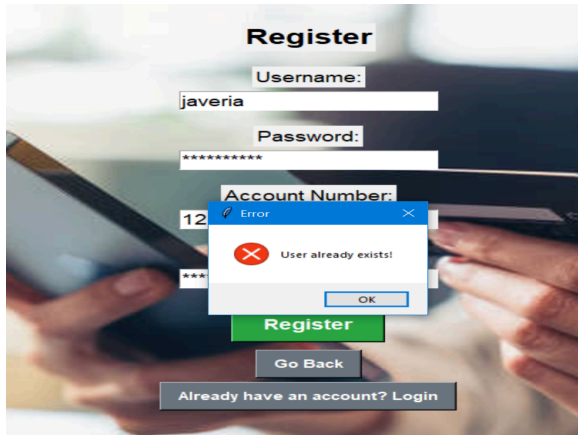
TC-R04



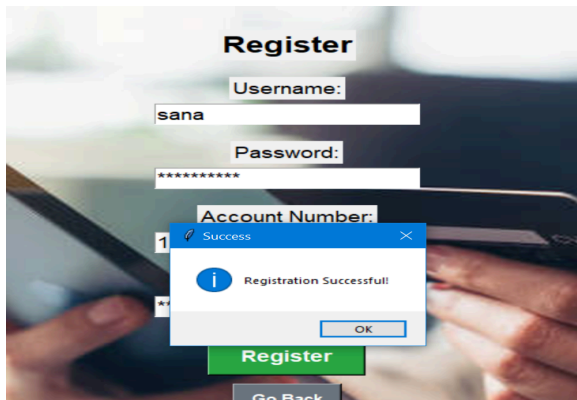
TC-R05



TC-R06



TC-R07



Functionality: Login

Code

```
def login():
    global current_user
    username = login_username_entry.get().strip()
    password = login_password_entry.get().strip()
    account_number = login_account_number_entry.get().strip()
    pin = login_pin_entry.get().strip()

    if username in users and users[username]["password"] == password and
    users[username]["account_number"] == account_number and users[username]["pin"] == pin:
        current_user = username
        messagebox.showinfo("Login", "Login Successful")
        switch_frame(dashboard_frame)
        update_dashboard()
```

```
else:
    messagebox.showerror("Error", "Invalid Credentials")
```

Summary Report

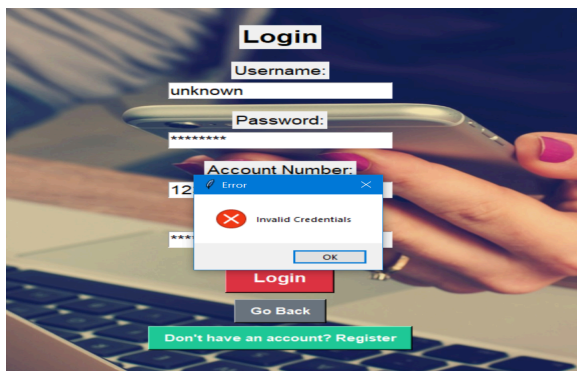
Test Case ID	Test Case Description	Test Steps	Test Data	Expected Result	Actual Result	Status
TC_L01	Login with invalid username	1. Enter non-existent username	Username: unknown	Error: Invalid Credentials	Error: Invalid Credentials	Pass
		2. Valid credentials	Password: pass123			
		3. Click Login	Account: 1234567890			
			PIN: 1234			
TC_L02	Login with incorrect password	1. Enter existing username	Username: sana	Error: Invalid Credentials	Error: Invalid Credentials	Pass
		2. Incorrect password	Password: wrongpass			
		3. Click Login	Account: 0987654321			
			PIN: 5678			
TC_L03	Login with incorrect account number	1. Enter existing username & password	Username: sana	Error: Invalid Credentials	Error: Invalid Credentials	Pass
		2. Wrong account number	Password: pass12345			
		3. Click Login	Account: 0000000000			
			PIN: 5678			
TC_L04	Login with incorrect PIN	1. Enter correct user/password/account	Username: sana	Error: Invalid Credentials	Error: Invalid Credentials	Pass
		2. Wrong PIN	Password: pass12345			
		3. Click Login	Account: 0987654321			
			PIN: 0000			
TC_L05	Valid Login	1. Enter all valid credentials	Username: sana	Login Successful	Login Successful	pass
		2. Click Login	Password: pass12345			
			Account: 0987654321			
			PIN: 5678			

Class Partitioning

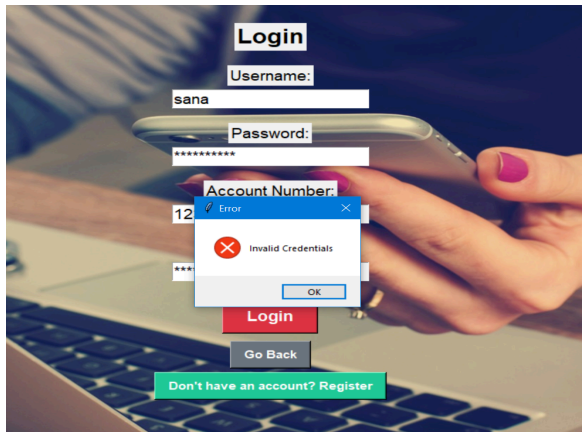
Field	Valid Equivalence Class	Invalid Equivalence Class
Username	Username exists in system (e.g., User123)	Username does not exist or is empty
Password	Correct password for the given username	Wrong password or empty password
Account Number	Correct account number for the given username	Wrong account number or empty account number
PIN	Correct PIN for the given username (4 digits)	Wrong PIN or empty PIN

Test Case

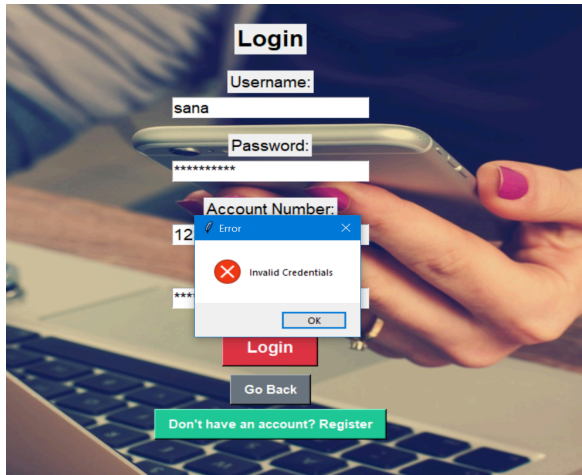
TC-L01



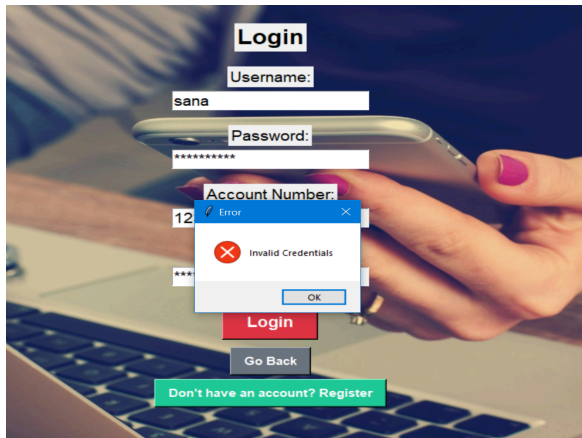
TC-LO2



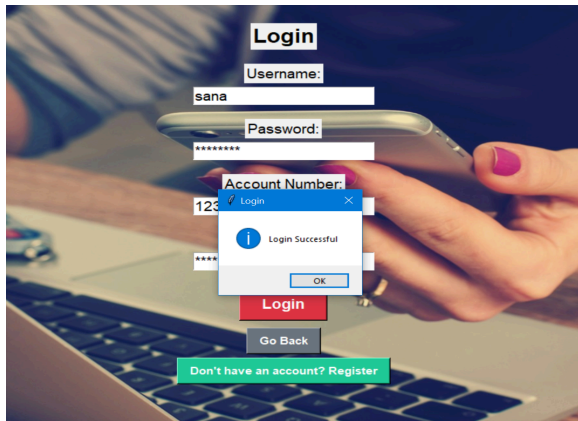
TC-LO3



TC-LO4



TC-LO5



Functionality: Deposit

Code

```
def deposit():
    amount = deposit_entry.get().strip()

    if not amount.isdigit() or int(amount) <= 0:
        messagebox.showerror("Error", "Enter a valid positive amount!")
        return
    if int(amount) > 100000:
        messagebox.showerror("Error", "Deposit amount exceeds the limit of $100,000!")
        return

    users[current_user]['balance'] += int(amount)
    messagebox.showinfo("Success", f"Deposited ${amount} successfully!")
    update_dashboard()
    deposit_entry.delete(0, tk.END)
```

Summary Report

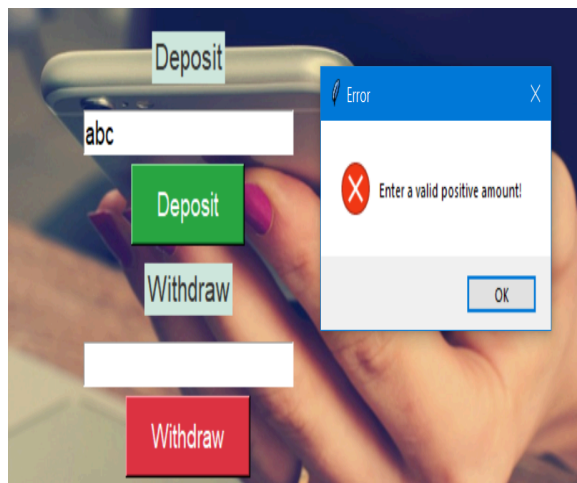
Test Case ID	Test Case Description	Test Steps	Test Data	Expected Result	Actual Result	Status
TC_D01	Deposit non-numeric value	1. Enter non-numeric value	Amount: abc	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	Pass
		2. Click Deposit				
TC_D02	Deposit negative value	1. Enter -1000	Amount: -1000	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	Pass
		2. Click Deposit				
TC_D03	Deposit zero	1. Enter 0	Amount: 0	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	Pass
		2. Click Deposit				
TC_D04	Deposit above limit	1. Enter 150000	Amount: 150000	Error: Deposit amount exceeds the limit!	Error: Deposit amount exceeds the limit!	Pass
		2. Click Deposit				
TC_D05	Valid Deposit	1. Enter 5000	Amount: 5000	Success: Deposited \$5000 successfully!	Success: Deposited \$5000 successfully!	
		2. Click Deposit				

Class Partitioning

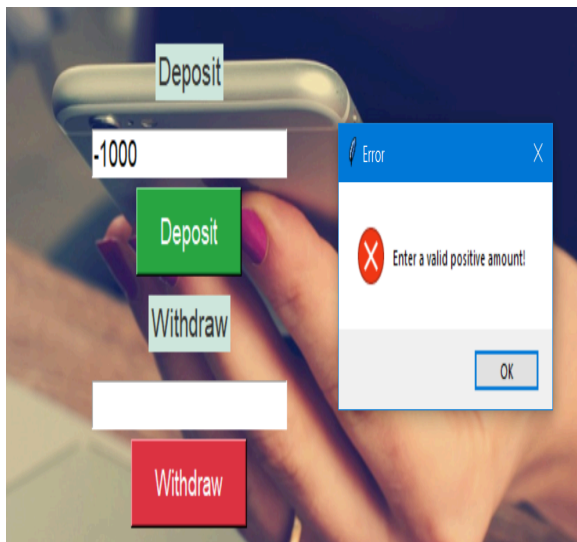
Field	Valid Equivalence Class	Invalid Equivalence Class
Amount	A positive numeric value $\leq 100,000$ (e.g., 1000)	- Empty input - Non-numeric input (e.g., "abc") - Negative number (e.g., -50) - Zero (0) - Amount greater than 100,000

Test Case

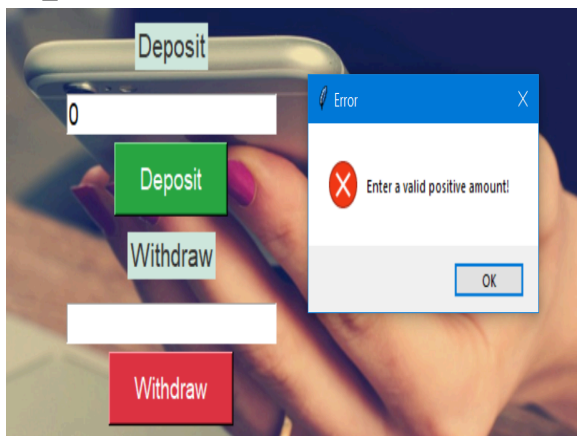
TC_D01



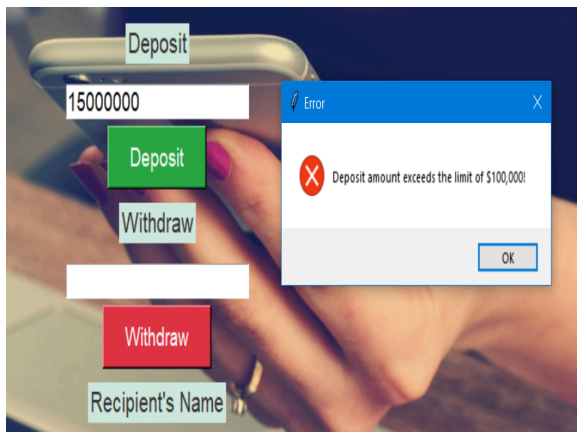
TC_D02



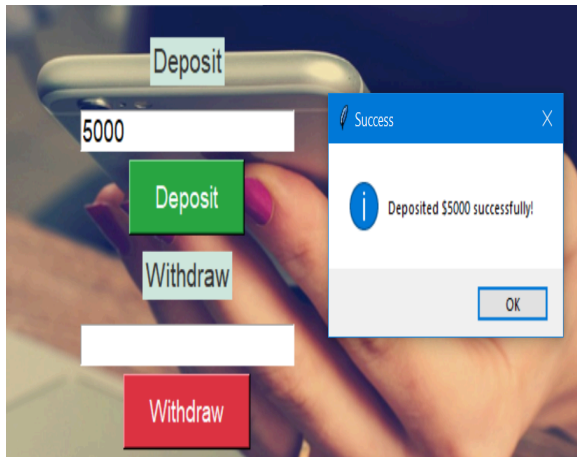
TC_D03



TC_D04



TC_D05



Functionality: Withdraw

Code

```
def withdraw():
    amount = withdraw_entry.get().strip()

    if not amount.isdigit() or int(amount) <= 0:
        messagebox.showerror("Error", "Enter a valid positive amount!")
        return
    amount = int(amount)
    if users[current_user]['balance'] >= amount:
        if amount > 50000:
            messagebox.showerror("Error", "Daily withdrawal limit exceeded ($50,000 max)!")
            return
        users[current_user]['balance'] -= amount
        messagebox.showinfo("Success", "Withdrawal Successful!")
    else:
        messagebox.showerror("Error", "Insufficient Balance!")

    update_dashboard()
    withdraw_entry.delete(0, tk.END)
```

Summary Report

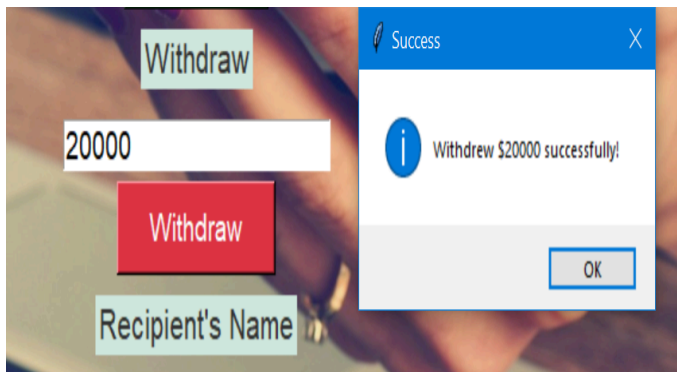
Test Case ID	Test Case Description	Test Steps	Test Data	Expected Result	Actual Result	Status
TC_W0_01	Valid Withdrawal	1. Log in	Amount: 20000, Balance: 50000	"Withdrawal Successful!"	"Withdrawal Successful!"	Pass
		2. Enter amount $\leq$ balance and $\leq$ 50000				
		3. Click Withdraw				
TC_W0_02	Amount Not Numeric	1. Log in	Amount: "twenty thousand"	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	Pass
		2. Enter non-numeric value				
		3. Click Withdraw				
TC_W0_03	Amount is Zero	1. Log in	Amount: 0	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	Pass
		2. Enter 0				
		3. Click Withdraw				
TC_W0_04	Amount is Negative	1. Log in	Amount: -1000	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	Pass
		2. Enter -1000				
TC_W0_05	Amount Exceeds Balance	1. Log in	Amount: 60000, Balance: 50000	Error: Insufficient Balance!	Error: Insufficient Balance!	Pass
		2. Enter amount $>$ balance				
		3. Click Withdraw				
TC_W0_06	Amount Exceeds Daily Limit	1. Log in 2. Enter amount $>$ 50000 $\leq$ balance 3. Click Withdraw	Amount: 60000, Balance: 100000	Error: Daily withdrawal limit exceeded (\$50,000 max)!	Error: Daily withdrawal limit exceeded (\$50,000 max)!	Pass

Class Partitioning

Field	Valid Equivalence Class	Invalid Equivalence Class
Amount	- Positive numeric value - $\leq$ Balance - $\leq$ 50,000	- Empty input - Non-numeric input (e.g., "abc") - Negative number (e.g., -20) - Zero (0) - Amount greater than balance - Amount greater than 50,000

Test Case

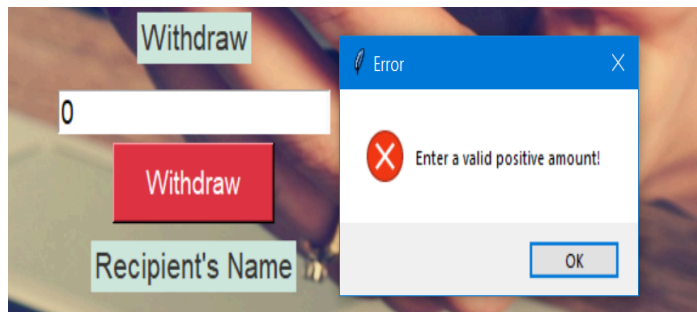
TC_W01



TC_W02



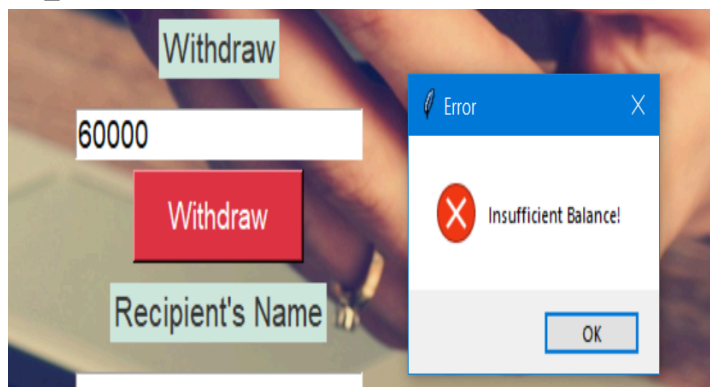
TC_W03



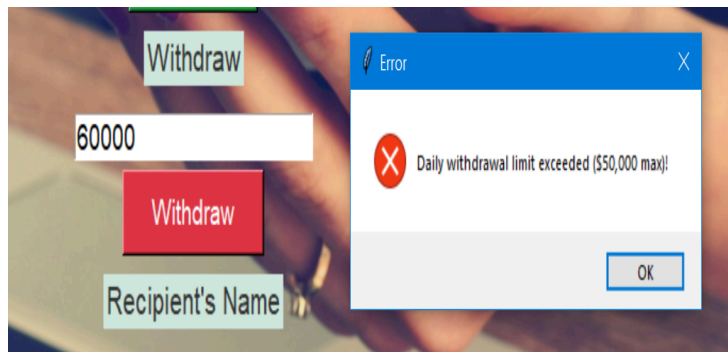
TC_W04



TC_W05



TC_W06



Functionality: Transfer

Code

```
def transfer():
    recipient = transfer_recipient_entry.get().strip()
    amount = transfer_amount_entry.get().strip()

    if recipient == current_user:
        messagebox.showerror("Error", "Cannot transfer to self!")
        return
    if recipient not in users:
        messagebox.showerror("Error", "Recipient not found!")
        return
    if not amount.isdigit() or int(amount) <= 0:
        messagebox.showerror("Error", "Enter a valid positive amount!")
        return
    if int(amount) > users[current_user]['balance']:
        messagebox.showerror("Error", "Limit Crossed!")
        return

    users[current_user]['balance'] -= int(amount)
    users[recipient]['balance'] += int(amount)
    messagebox.showinfo("Success", f"Transferred ${amount} to {recipient} successfully!")
    update_dashboard()
    transfer_recipient_entry.delete(0, tk.END)
    transfer_amount_entry.delete(0, tk.END)
```

Summary Report

Test Case ID	Test Case Description	Test Steps	Test Data	Expected Result	Actual Result	Status
TC_T01	Recipient Field is Empty	1. Log in as current_user	To User: (empty), Amount: 5000	Error: Recipient not found!	Error: Recipient not found!	Pass
		2. Leave recipient field empty				
		3. Enter valid amount				
		4. Click Transfer				
TC_T02	Transfer to Self Not Allowed	1. Log in as current_user	To User: current_user, Amount: 3000	Error: Cannot transfer to self!	Error: Cannot transfer to self!	Pass
		2. Enter your own username as recipient				
		3. Click Transfer				
TC_T03	Recipient Not Found	1. Log in as current_user	To User: unknown_user, Amount: 4000	Error: Recipient not found!	Error: Recipient not found!	Pass
		2. Enter non-existent username				
		3. Enter valid amount				
		4. Click Transfer				
TC_T04	Invalid Amount (Non-digit)	1. Log in as current_user	To User: sana, Amount: -1000	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	Pass
		2. Enter valid recipient				
		3. Enter amount as text (e.g., "five hundred")				
		4. Click Transfer				
TC_T05	Invalid Amount (Zero or Negative)	1. Log in as current_user	To User: sana, Amount: 0	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	Pass
		2. Enter valid recipient				
		3. Enter amount as 0 or negative				
		4. Click Transfer				

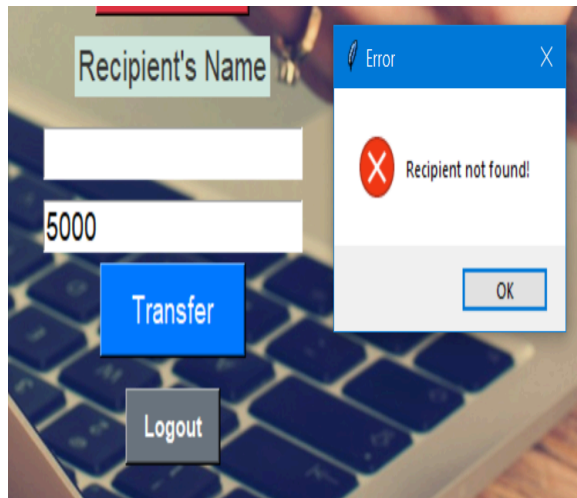
TC_T06	Limit Cross	1. Log in as current_user	To User: sana, Amount: 20000	Error: Limit Crossed!	Error: Limit Crossed!	Pass
		2. Enter valid recipient				
		3. Enter amount greater than current_user's balance				
		4. Click Transfer				
TC_T07	Insufficient Balance	1. Log in as current_user	To User: sana, Amount: 980000	Error: Insufficient balance!	Error: Insufficient balance!	Pass
		2. Enter valid recipient				
		3. Enter amount ≤ balance				
		4. Click Transfer				
TC_T08	Valid Transfer	1. Log in as current_user	To User: sana, Amount: 980000	Transfer successful, balance updated	Transfer successful, balance updated	Pass
		2. Enter valid recipient				
		3. Enter amount ≤ balance				
		4. Click Transfer				

Class Partitioning

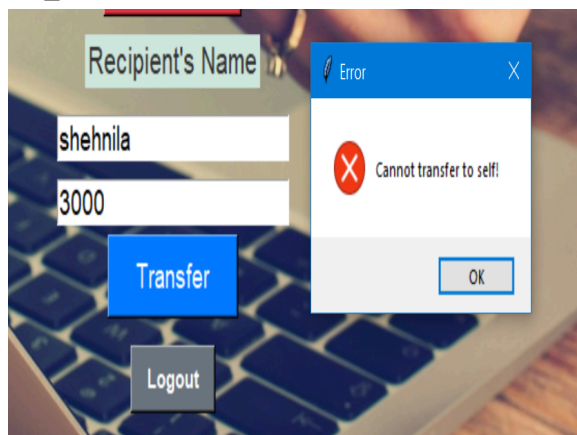
Field	Valid Equivalence Class	Invalid Equivalence Class
Recipient Username	Existing user other than current user	- Same as current user (self-transfer)
		- Username not found
Amount	Positive numeric value ≤ current balance	- Empty input
		- Non-numeric input (e.g., "abc")
		- Negative number (e.g., -10)
		- Zero (0)
		- Amount greater than current balance

Test Case

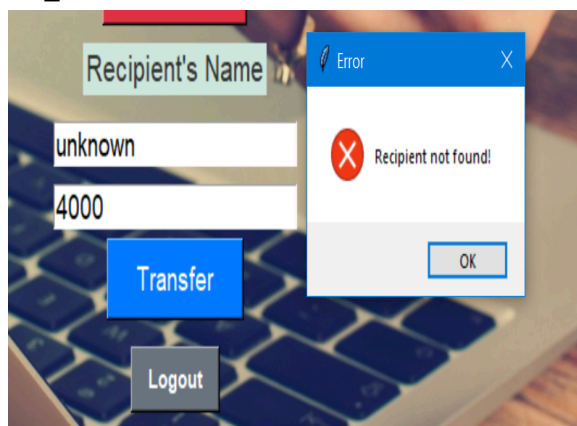
TC_01



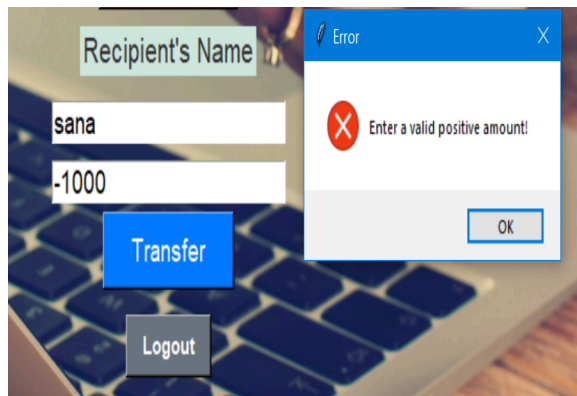
TC_02



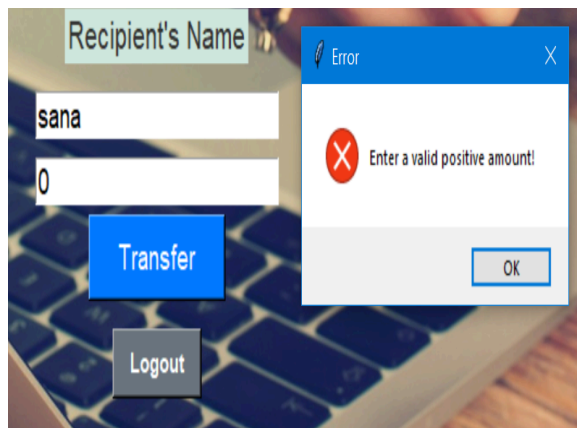
TC_03



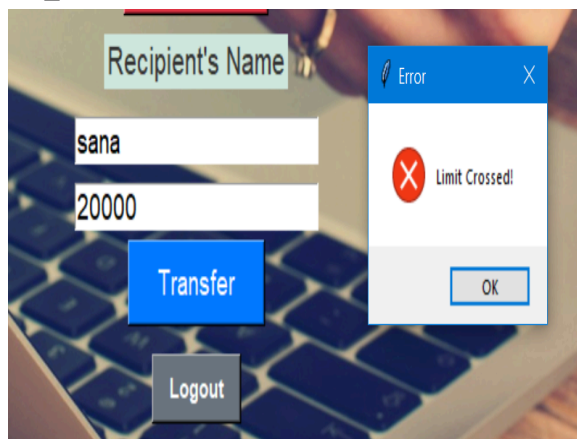
TC_04



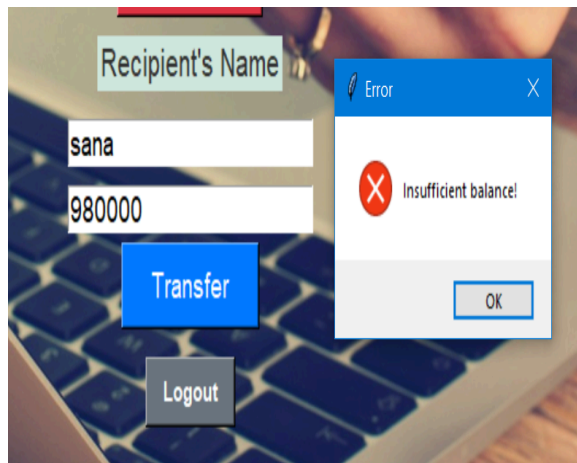
TC_05



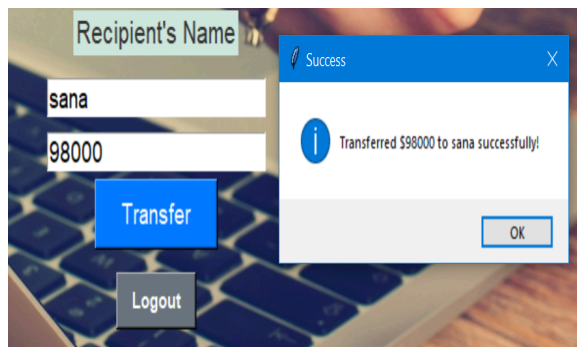
TC_06



TC_07



TC_08



Also answer the following questions

i. Draw a comparison of selected techniques with all other ones.

Equivalence Class Partitioning is a black box testing methodology for separating input data into valid and invalid partitions to reduce the number of test cases while still providing effective coverage. When compared with Boundary Value Analysis (BVA), where it is used for testing values only at the edge of input ranges (i. e. minimum and maximum limits), ECP tests a single (representative) value from each partition instead of only focusing on a boundary point. When compared with Decision Table Testing, where multiple input combinations can be tested and complex business logic is tested, ECP is simpler and more efficient when testing simple input validation. When compared with State Transition Testing, where it is used to validate the behavior of the system when some state changes (e. g. a new start or new end state) vs. existing state, ECP is focused on just classifying the inputs without considering change in the system behavior. Lastly, when compared to Use Case Testing, where all user actions and flows are validated, ECP is focused on simply dividing and testing sets of inputs instead of user actions or flows; it is therefore best used when validation is highly dependent on input variation.

ii. Why you choose this technique?

The chosen ECP is Equivalence Class Partitioning (ECP) which allows reducing overall number of test cases while maintaining good coverage of the behaviour of the application. ECP works very effectively when the input data can be broken down into clear valid and invalid groups, as it was the case with my project. With ECP easy errors related to different types of inputs can be detected very quickly without need of testing every single value. So testing process becomes faster and more efficient, because all important input conditions are checked without redundancy.

iii. On what level we can apply black box and why?

Black Box testing (including techniques such as ECP) can be done both at the System Level and at the Acceptance Level: At the System Level, it is known that the whole application works like expected. So it is done to test the whole functionality without checking the inner code. At the Acceptance Level, it is known that it is done to validate whether the system meets the business requirements and the user's expectations. So black box testing would suit best here as one has only one focus, on the input and output behavior, rather than on the internal way the system was built.

